

Öğrenci Servis Takip ve Yönetim Sistemi - Teknik Tasarım Belgesi

Sürüm: 1.0.0

Durum: Tasarım Aşaması

Son Güncelleme: 29 Aralık 2025

1. Proje Özeti ve Kapsam

Bu proje, okul servislerinin yönetimini dijitalleştiren; yönetici, şoför ve veli arasındaki bilgi akışını sağlayan web tabanlı bir yönetim sistemidir. Sistem, araç atamaları, günlük öğrenci takibi (bindi/indi durumu) ve genel servis yönetimini hedefler. Gelecekte mobil uygulama ve canlı konum takibi özelliklerine genişletilebilir bir yapıda tasarlanmıştır.

2. Sistem Mimarisi

Sistem, **MVC (Model-View-Controller)** mimari deseni üzerine kurulmuş olup, katmanlı bir yapı izlemektedir.

2.1. Katmanlı Yapı (Layered Architecture)

Katman	Teknoloji	Görev
Sunum Katmanı (Frontend)	HTML5, CSS3, Vanilla JS	Kullanıcı arayüzü, Admin (Sürükle-Bırak), Şoför ve Veli panelleri.
Uygulama Katmanı (Backend)	PHP 7.4+ (Custom MVC)	İş mantığı, API uç noktaları, JWT Kimlik Doğrulama, Yönlendirme.
Veri Katmanı (Database)	MySQL / MariaDB	İlişkisel veritabanı, veri saklama ve sorgulama.

2.2. Veri Akış Şeması

- İstemci (Client):** HTTP isteği gönderir (JSON formatında).
- Router:** İsteği karşılar ve ilgili Controller'a yönlendirir.
- Middleware:** JWT Token kontrolü ve Rol (Admin/Driver/Parent) yetkilendirmesini yapar.
- Controller:** İsteği işler, Model ile veri alışverişi yapar.
- Model:** Veritabanı (PDO) ile güvenli sorgular çalıştırır.
- Response:** İşlenen veriyi JSON formatında istemciye döner.

3. Dosya ve Klasör Organizasyonu

Proje, güvenlik ve sürdürülebilirlik açısından `public` klasörü dışarıya açık, çekirdek dosyalar ise korumalı olacak şekilde yapılandırılmıştır.

```
/
├── public/                                # Web sunucusunun kök dizini (Erişime açık)
│   ├── index.php                         # Uygulamanın giriş kapısı (Front Controller)
│   ├── assets/                          # Statik dosyalar
│   │   ├── css/                        # (admin.css, driver.css, parent.css)
│   │   ├── js/                        # (admin.js, driver.js, api.js, auth.js)
│   │   ├── lib/                       # (sortable.min.js, jwt-decode.min.js)
│   │   └── images/
│   └── .htaccess                        # URL yönlendirme kuralları
├── app/                                  # Uygulama mantığı (Erişime kapalı)
│   ├── Core/                          # Çekirdek sınıflar (Router, Database, JWT, Respon
│   ├── Controllers/                  # (AuthController, AdminController, StudentControl
│   ├── Models/                      # (User, Driver, Student, Service, Attendance...)
│   ├── Middleware/                  # (AuthMiddleware, RoleMiddleware)
│   └── Views/                        # HTML Şablonları (layouts, admin, driver, parent)
├── config/                             # Konfigürasyon dosyaları
│   ├── database.php                 # DB bağlantı ayarları
│   ├── constants.php               # Sabitler
│   └── jwt_config.php              # Gizli anahtarlar
├── api/                                # API Uç Noktaları (Router tarafından çağrılır)
├── database/                          # Veritabanı şemaları ve seed dosyaları
└── logs/                             # Hata kayıtları
```

4. Veritabanı Tasarımı (Schema)

Veritabanı 3. normal formda (3NF) normalize edilmiş ilişkisel tablolardan oluşur.

4.1. Temel Tablolar

`users` (Kullanıcılar)

Sisteme giriş yapabilen tüm aktörleri tutar.

- **Roller:** `admin`, `parent`
- **İlişkiler:** `drivers` ve `students` tabloları ile ilişkilidir.

`drivers` (Şoförler)

Servis sürücülerinin detaylarını tutar.

- `user_id` : Opsiyonel olarak `users` tablosuna bağlanır (Şoför paneline giriş yapacaksa).

- `license_number`, `status` gibi spesifik alanlar içerir.

`vehicles` (Araçlar)

Fiziksel araç envanterini tutar.

- `plate_number` (Unique), `capacity`, `status`.

`services` (Servis Rotaları)

Mantıksal servis rotalarını tanımlar (Örn: "Sabah - Kadıköy Hattı").

`students` (Öğrenciler)

Taşınan öğrencilerin bilgilerini tutar.

- `parent_user_id` : Ebeveyn ile ilişkilendirir.
- `service_id` : Hangi servise ait olduğunu belirtir.

4.2. Operasyonel Tablolar

`assignments` (Atamalar)

Sistemin kalbidir. Şoför, Araç ve Servis üçlüsünü belirli bir tarih aralığı için eşleştirir.

- **Constraint:** Bir şoför/araç aynı zaman diliminde birden fazla aktif atamaya sahip olamaz.

`daily_attendance` (Günlük Yoklama)

Günlük bindi/indi verilerini tutar.

- `status` : `boarded` (bindi), `not_boarded` (binmedi), `absent` (gelmedi).

`location_tracking` (Konum - Gelecek Faz)

Araçların anlık konum geçmişini saklamak için tasarlanmıştır.

5. API Spesifikasyonu

Tüm API yanıtları standart JSON formatındadır.

5.1. Kimlik Doğrulama (Auth)

- `POST /api/auth/login` : Kullanıcı girişi (JWT Token döner).
- `GET /api/auth/me` : Token ile mevcut kullanıcı bilgilerini getirir.

5.2. Yönetici (Admin) İşlemleri

- `GET /api/vehicles` : Araç listesi.

- `POST /api/vehicles` : Yeni araç ekleme.
- `POST /api/vehicles/reorder` : Araçları sürükle-bırak sırasına göre güncelleme.
- `POST /api/assignments` : Şoför-Araç-Servis ataması yapma.

5.3. Şoför (Driver) İşlemleri

- `GET /api/driver/assignment` : Şoföre atanmış aktif görevi getirir.
- `GET /api/driver/daily-list` : O günkü öğrenci listesini getirir.
- `POST /api/driver/attendance` : Öğrenci durumunu (bindi/binmedi) günceller.

5.4. Veli (Parent) İşlemleri

- `GET /api/parent/child-info` : Çocuğun servis, araç ve şoför bilgilerini getirir.
- `GET /api/parent/attendance-history` : Geçmişe dönük servis kullanım raporu.

6. Güvenlik ve En İyi Uygulamalar

Sistem güvenliği için aşağıdaki protokoller zorunludur:

1. **JWT (JSON Web Token):** Oturum yönetimi için sunucu taraflı session yerine stateless JWT kullanılır.
2. **PDO & Prepared Statements:** Tüm veritabanı sorguları SQL Injection saldırılarına karşı parameterize edilir.
3. **Parola Güvenliği:** Parolalar veritabanında asla düz metin saklanmaz; `password_hash()` (Bcrypt/Argon2) kullanılır.
4. **XSS & CSRF:** Çıktılar `htmlspecialchars()` ile temizlenir. Form işlemlerinde CSRF token veya origin kontrolü yapılır.
5. **Dizin Güvenliği:** `.htaccess` ile izin listeleme (`Options -Indexes`) kapatılır ve hassas dosyalara doğrudan erişim engellenir.

7. Geliştirme Yol Haritası (Roadmap)

Faz 1: MVP (Minimum Viable Product)

- [x] Veritabanı mimarisinin kurulması.
- [] Temel MVC framework ve Routing yapısının kodlanması.
- [] Kullanıcı Giriş/Çıkış (Auth) mekanizması.
- [] Admin: Araç, Şoför, Öğrenci CRUD işlemleri.
- [] Şoför: Günlük listeyi görme ve yoklama alma.

Faz 2: Gelişmiş Yönetim & UI

- [] Admin: Sürükle-Bırak (Drag & Drop) ile atama arayüzü.
- [] Veli Paneli: Detaylı çocuk ve servis bilgisi görüntüleme.
- [] Raporlama: PDF formatında servis listesi çıktısı.

Faz 3: Mobil Entegrasyon & Canlı Takip

- [] API'nin mobil uygulama için optimize edilmesi.
- [] Canlı konum verisinin veritabanına işlenmesi.
- [] Veliler için anlık bildirimler (Push Notifications).

8. Teknoloji Yığını (Stack)

- **Backend:** PHP 7.4+
- **Database:** MySQL 5.7+ / MariaDB 10.3+
- **Frontend:** HTML5, CSS3, JavaScript (ES6+)
- **Libraries:**
 - `firebase/php-jwt` (Backend JWT yönetimi için)
 - `SortableJS` (Frontend sürükle-bırak işlemleri için)